Rendre compatible votre interface de vue avec une interface standard grâce au pattern Adapter

1. Contexte

Vous avez développé, dans les étapes précédentes du TD MVC :

- un modèle complet du jeu (Deck, Card, Rank, Suit, Hand, Player),
- un contrôleur qui gère la machine à états,
- **votre propre interface de vue**, avec un nommage qui peut varier selon les étudiants,
- une vue console qui implémente cette interface.

Cette **interface standardisée** de vue est fournie plus bas afin de permettre le branchement d'une vue graphique Swing.

Votre interface personnelle peut avoir des noms différents.

L'objectif de cette partie du TD est de rendre **votre interface de vue actuelle compatible** avec l'interface standardisée, **sans renommer vos méthodes**, et **sans modifier votre contrôleur MVC**.

---

2. Interface standardisée de vue (à utiliser avec la vue Swing)

```
public interface GameViewable {

    void promptForPlayerName();
    void promptForFlip();
    void promptForNewGame();

    void showPlayerName(int index, String name);
    void showFaceDownCardForPlayer(int index, String name);
    void showCardForPlayer(int index, String name, String rank, String suit);

    void showWinner(String name);

    void setController(GameController gc);
}
```

Cette interface correspond **strictement** à ce qu'attend la vue Swing.

---

3. Votre interface actuelle (variable selon les étudiants)

Chaque groupe possède déjà une interface définie au début du TD, par exemple :

```
public interface JeuVisible {
    void demandeNomJoueur();
    void demandeRetournerCarte();
    void demandeNouveauJeu();

    void montreNomJoueur(int index, String nom);
    void montreFaceCacheJoueur(int index, String nom);
    void montreCarteJoueur(int index, String nom, String rang, String
couleur);

    void montreGagnant(String nom);

    void setController(GameController gc);
}
```

Les noms sont évidemment différents dans votre projet, et c'est précisément le but de ce TD :

votre code actuel doit rester tel quel.

---

4. Problème à résoudre

Le contrôleur a été écrit pour dépendre de **votre interface personnelle** :

```
public class GameController {
    private JeuVisible vue;
}
```

Mais la vue Swing dépend de GameViewable, avec une API différente.

Deux interfaces différentes → incompatibilité directe.

On ne renomme rien.

On ne modifie pas le contrôleur.

On n'étend pas GameSwing.

→ On applique le **pattern Adapter**.

---

5. Objectif

Créer un adapter qui :

1. **implémente votre propre interface de vue**,
2. **contient une référence vers l'interface standard** (GameViewable),
3. **redirige chaque appel vers l'équivalent standardisé**.

Cela permet :

- de continuer à utiliser votre contrôleur MVC tel qu'il est,
- de brancher la vue Swing immédiatement,
- de respecter le nommage que vous avez choisi au début du TD.

---

6. Implémentation de l'Adapter

Créer une classe :

```
public class GameSwingAdapter implements JeuVisible {
```

Elle stocke l'instance de la vue Swing :

```
private final GameViewable target;

public GameSwingAdapter(GameViewable target) {
    this.target = target;
}
```

Puis les correspondances :

```
@Override
public void demandeNomJoueur() {
    target.promptForPlayerName();
}

@Override
public void demandeRetournerCarte() {
    target.promptForFlip();
}

@Override
public void demandeNouveauJeu() {
    target.promptForNewGame();
}
```

```
@Override
public void montreNomJoueur(int index, String nom) {
    target.showPlayerName(index, nom);
}

@Override
public void montreFaceCacheJoueur(int index, String nom) {
    target.showFaceDownCardForPlayer(index, nom);
}

@Override
public void montreCarteJoueur(int index, String nom, String rang, String
couleur) {
    target.showCardForPlayer(index, nom, rang, couleur);
}

@Override
public void montreGagnant(String nom) {
    target.showWinner(nom);
}

@Override
public void setController(GameController gc) {
    target.setController(gc);
}
```

Adaptez les correspondances de nom des méthodes.

---

7. Intégration dans votre programme MVC

Dans votre main :

```
GameViewable swing = new GameSwing();
JeuVisible vue = new GameSwingAdapter(swing);

GameController gc = new GameController(deck, vue);
vue.setController(gc);

gc.run();
```

Cela permet au contrôleur de continuer à voir **son** interface, tout en utilisant la vue Swing standard.